

Pagination Strategies

Java Backend Interview Reference

Why Pagination Matters

Pagination controls how large datasets are split into pages for API responses. Without it, fetching millions of rows kills performance, wastes bandwidth, and crashes clients. The choice of strategy affects consistency, performance, deep-page scalability, and sort flexibility.

1. Offset-Based Pagination (LIMIT / OFFSET)

Concept

The client passes a page number and size. SQL uses LIMIT/OFFSET. Most common, easiest to implement.

Attribute	Detail
SQL	<pre>SELECT * FROM orders ORDER BY created_at DESC LIMIT 20 OFFSET 40;</pre>
Spring Data	<pre>Pageable p = PageRequest.of(page, size, Sort.by("createdAt").descending()); Page<Order> result = repo.findAll(p);</pre>
Response	<pre>{ "data": [...], "page": 3, "size": 20, "total": 500, "totalPages": 25 }</pre>
Pros	Simple to implement • Random page access • Works with any sort • UI page numbers easy
Cons	Slow at deep pages (OFFSET scans all prior rows) • Inconsistent if rows insert/delete (page drift) • COUNT(*) expensive • Not suitable for real-time streams
Best For	Admin dashboards • Static/infrequently changing data • Small to medium datasets (<100k rows)

2. Keyset / Cursor-Based Pagination

Concept

Instead of OFFSET, the client passes the last seen value of the sort key. The DB uses a WHERE clause to seek efficiently. Also called "seek method" or "cursor pagination".

Attribute	Detail
SQL	<pre>-- First page SELECT * FROM orders ORDER BY created_at DESC, id DESC LIMIT 20; -- Next page (pass last row's created_at + id) SELECT * FROM orders WHERE (created_at, id) < ('2024-01-10 12:00:00', 9823) ORDER BY created_at DESC, id DESC LIMIT 20;</pre>
Spring Data	<pre>@Query("SELECT o FROM Order o WHERE (o.createdAt < :lastDate OR (o.createdAt = :lastDate AND o.id < :lastId)) ORDER BY o.createdAt DESC, o.id DESC") Page<Order> findNextPage(@Param("lastDate") LocalDateTime lastDate, @Param("lastId") Long lastId, Pageable p);</pre>
Response	<pre>{ "data": [...], "nextCursor": "eyJpZCI6OTgyMywgImNyZWZ0ZWQiOiIyMDI0LTAxLTUwIn0=" }</pre>
Pros	O(1) seek with index • Stable even if rows inserted/deleted • Scales to millions of rows • Ideal for feeds/streams
Cons	No random page access • Must include sort key in cursor • Harder to implement multi-column sort • No total count easily
Cursor Encoding	Base64-encode {id, timestamp} to prevent client tampering. JWT signing optional for security.
Best For	Social feeds • Chat history • Activity logs • High-volume append-heavy tables • Infinite scroll

3. Cursor Pagination (Opaque Token)

A variant where the cursor is fully opaque — server encodes all state needed to resume. Client cannot interpret or manipulate it. Supported by GraphQL Relay spec and many REST APIs.

Field	Description
endCursor	Opaque string representing position after last item returned

Field	Description
hasNextPage	Boolean — whether more items exist after this page
hasPreviousPage	Boolean — for backward navigation support
startCursor	Position of first item returned (for backward nav)

```
// GraphQL Relay example { orders(first: 20, after: "Y3Vyc29yMQ==") { edges { node { id amount } cursor } pageInfo { endCursor hasNextPage } }}
```

4. Time-Based Pagination

Use timestamp ranges instead of offsets or keys. Common for time-series data, logs, and events.

Type	Example	Use Case
Sliding Window	WHERE created_at BETWEEN :from AND :to ORDER BY created_at	Logs, metrics, analytics
Since/Until	WHERE created_at > :since LIMIT 100	Activity feeds, webhooks
Epoch Cursor	WHERE event_time > :lastEventTime LIMIT 50	Event sourcing, CDC

5. Search-Engine Pagination (Elasticsearch / Solr)

For full-text search results. Elasticsearch supports from/size (offset), search_after (keyset), and scroll API (batch export).

Method	API	Use Case	Limit
from/size	{"from":40,"size":20}	Simple search UI	Max 10,000 (index.max_result_window)
search_after	{"search_after":["1704067200000,9823"],"size":20}	Deep pagination, real-time	Requires sort + tiebreaker
scroll	GET /idx/_search?scroll=2m	Bulk export, reindex	Not for real-time; snapshot-based
PIT + search_after	POST /_pit?keep_alive=1m then search_after	Consistent deep pages	Best practice for ES 7.10+

Strategy Comparison Table

Strategy	Deep Pages	Consistency	Random Access	Count	Best For
Offset	Slow O(n)	Poor	Yes	Yes	Admin UIs, static data
Keyset	Fast O(1)	Good	No	No	Feeds, streams, logs
Cursor (opaque)	Fast O(1)	Good	No	No	GraphQL, REST APIs
Time-based	Fast	Good	Range only	Estimate	Time-series, events
ES from/size	Slow	Poor	Yes	Yes	Small result sets
ES search_after	Fast	Good (PIT)	No	Yes	Large search results

Java / Spring Boot Implementation Patterns

PageRequest with Sort

```
// Controller @GetMapping("/orders") public ResponseEntity<PageResponse<OrderDTO>> getOrders(
@RequestParam(defaultValue = "0") int page, @RequestParam(defaultValue = "20") int size,
@RequestParam(defaultValue = "createdAt") String sortBy, @RequestParam(defaultValue = "DESC") String dir) {
Sort sort = Sort.by(Sort.Direction.fromString(dir), sortBy); Pageable pageable = PageRequest.of(page, size, sort);
return ResponseEntity.ok(orderService.getOrders(pageable)); }
```

Custom Cursor Response Wrapper

```
public record CursorPage<T>( List<T> data, String nextCursor, // base64({id,timestamp}) boolean hasMore, int size ) { public static <T> CursorPage<T> of(List<T> data, String cursor, int size) { return new CursorPage<>(data, cursor, data.size() == size, size); } }
```

Keyset Service Method

```
public CursorPage<OrderDTO> getOrdersAfter(String cursor, int size) { CursorData cd = decodeCursor(cursor); // decode base64 -> {id, timestamp} List<Order> orders = orderRepo.findByCreatedAtBeforeOrEqualAndIdLessThan(cd.timestamp(), cd.id(), PageRequest.of(0, size + 1)); boolean hasMore = orders.size() > size; List<Order> page = hasMore ? orders.subList(0, size) : orders; String nextCursor = hasMore ? encodeCursor(page.get(page.size()-1)) : null; return CursorPage.of(toDTO(page), nextCursor, size); }
```

Cursor Encoding / Decoding

```
private String encodeCursor(Order o) { String raw = o.getId() + ":" + o.getCreatedAt().toEpochMilli(); return Base64.getUrlEncoder().encodeToString(raw.getBytes()); } private CursorData decodeCursor(String cursor) { String raw = new String(Base64.getUrlDecoder().decode(cursor)); String[] parts = raw.split(":"); return new CursorData(Long.parseLong(parts[0]), Instant.ofEpochMilli(Long.parseLong(parts[1]))); }
```

Database Index Design for Pagination

Strategy	Required Index	SQL Hint
Offset on createdAt	CREATE INDEX idx_orders_created ON orders(created_at DESC);	Sort key must be indexed
Keyset (createdAt + id)	CREATE INDEX idx_orders_cs ON orders(created_at DESC, id DESC);	Composite — order matters
Search + keyset	CREATE INDEX idx_orders_status_cs ON orders(status, created_at DESC, id DESC);	Filter col first, then sort cols
UUID keyset	CREATE INDEX idx_orders_uuid ON orders(id); -- UUID v7 (time-sortable)	UUID v7 is time-ordered, safe for keyset

Edge Cases & Production Gotchas

Problem	Explanation	Fix
Page drift (offset)	Row inserted on page 2 while user reads page 3 — they skip or see duplicate	Switch to keyset; or accept and warn users
Duplicate sort key (keyset)	Two rows with same timestamp — cursor ambiguous	Always include unique tiebreaker (id) in cursor
Total count is slow	COUNT(*) on large tables with WHERE can be 100ms+	Use approximate count (EXPLAIN rows estimate), cache count, or drop total from response
Client manipulates cursor	Client decodes base64 and forges cursor position	HMAC-sign cursor or use opaque server-stored cursor
Sort column not indexed	Keyset still does full scan without index on sort col	Always index sort columns; use EXPLAIN to verify Index Scan
Deep ES offset	ES from/size beyond 10k throws exception	Switch to search_after + PIT; raise max_result_window only if necessary
Null sort values	NULLs sort inconsistently across DBs	Use NULLS LAST/FIRST, or filter out NULLs, or use a sentinel value

API Design Best Practices

Practice	Example
Return pagination metadata	{ "data":[...], "pagination": { "nextCursor":"abc", "hasMore":true, "size":20 } }
Support both page and cursor	Allow ?page=2&size=20 for offset; ?cursor=abc&size=20 for keyset

Practice	Example
Max page size cap	<code>@Min(1) @Max(100) @RequestParam(defaultValue="20") int size</code>
Default sort	Always sort by at least one indexed column — never paginate unsorted results
Link header (REST)	Link: <code></api/orders?cursor=abc>; rel="next", </api/orders>; rel="first"</code>
Idempotent cursors	Cursor must represent position, not time — re-requesting same cursor returns same data

Realistic Use Cases

App	Strategy	Reason
E-commerce product list	Offset	Filtering, sorting, page numbers needed for SEO
Twitter/Instagram feed	Keyset (createdAt, id)	Infinite scroll, append-heavy, consistency critical
Order history	Keyset or cursor	User expects stable ordering; deep pages possible
Admin transaction log	Offset with date filter	Random access, moderate volume, date range filter
Chat message history	Keyset (messageId)	Append-only, always sorted, high volume
Search results	ES from/size < 1k, search_after beyond	Full-text score sort, then relevance keyset
Analytics export	Keyset batch / scroll	Bulk, no UX — just throughput; cursor avoids drift
Real-time event stream	Time-based cursor (since)	Poll for events after last seen timestamp

Interview Q&A;

Q1: What is the main performance problem with OFFSET pagination?

OFFSET forces the DB to scan and discard all prior rows before returning the requested page. For OFFSET 10000 LIMIT 20, DB processes 10,020 rows just to return 20. Keyset pagination avoids this by using a WHERE clause with an indexed seek.

Q2: When would you prefer offset over keyset pagination?

When you need random page access (jump to page 15), total item count, UI page numbers, or support flexible multi-column sorting that changes per user. Offset is also simpler to implement for relatively small datasets (<100k rows).

Q3: How do you ensure cursor stability when rows can be deleted?

Use keyset with a unique immutable identifier (like id). Even if rows are deleted, the WHERE id < :lastId clause still returns the next valid batch correctly. Offset pagination would silently skip rows on deletion.

Q4: How would you paginate a search endpoint backed by Elasticsearch?

For up to ~1000 results: from/size is acceptable. For deeper pagination: use search_after with a Point-in-Time (PIT) context to get consistent snapshot pagination. Never exceed the default max_result_window (10,000) with from/size.

Q5: How do you prevent cursor forgery?

HMAC-sign the cursor payload using a server-side secret: Base64(JSON + HMAC-SHA256). Verify the signature on each request before using the cursor values in queries. Alternatively, store cursor state server-side in Redis with a UUID token.

Q6: What index is required for keyset pagination (createdAt DESC, id DESC)?

A composite index: CREATE INDEX idx ON table(createdAt DESC, id DESC). Column order in the index must match the ORDER BY clause. The WHERE clause uses row value comparison: WHERE (createdAt, id) < (:lastDate, :lastId).

Q7: How do you return total count efficiently with pagination?

Options: (1) Exact COUNT(*) — accurate but slow at scale; (2) Cached count — update on insert/delete with a counter table; (3) Approximate count from EXPLAIN or pg_stat; (4) Omit count — many modern APIs (Twitter, GitHub) return only hasMore flag instead.

Q8: How does GraphQL Relay-style pagination work?

Relay spec uses edges/node/cursor structure. Each item (node) has an opaque cursor. pageInfo contains endCursor (last item cursor), hasNextPage, hasPreviousPage, startCursor. Clients use first+after for forward nav, last+before for backward.

Q9: What is UUID v7 and why is it good for pagination?

UUID v7 is time-sortable — the first 48 bits encode millisecond timestamp. Unlike UUID v4 (random), UUIDv7 rows are naturally ordered by insertion time. This makes them usable as keyset cursor fields without storing a separate timestamp.

Q10: How would you paginate a multi-tenant SaaS query?

Always filter by tenantId first (leftmost index column), then sort columns: `CREATE INDEX ON orders(tenant_id, created_at DESC, id DESC)`. Cursor includes tenantId implicitly from authenticated user context, never from client input.